Infrastructure and Dependency Management Policy

Transparency Hub Network Version: 1.0 **Effective Date:** March 5, 2026 **Last Updated:** March 5, 2026 **Owner:** Engineering Team **Classification:** Internal

---

1. Purpose

This policy defines how Transparency Hub Network manages, secures, monitors, and maintains its cloud infrastructure and software dependencies. It ensures that all infrastructure components are properly configured, patched, and monitored, and that all third-party software dependencies are vetted, tracked, and kept secure throughout their lifecycle.

---

2. Scope

This policy applies to:

- All cloud infrastructure on Microsoft Azure
- All application runtime environments
- All third-party software packages and libraries (frontend and backend)
- All external service integrations and APIs
- Development, staging, and production environments
- CI/CD pipelines and build tooling
- All personnel responsible for infrastructure and development

---

3. Infrastructure Overview

3.1 Architecture

Transparency Hub Network operates on a **Platform-as-a-Service (PaaS)** architecture on Microsoft Azure, minimizing infrastructure management overhead while maintaining security and scalability.

Component	Azure Service	Purpose	Environment
Frontend Application	Azure Static Web Apps	Next.js admin dashboard	Staging + Production
Public Website	Azure Static Web Apps	Marketing and public pages	Production
Backend API	Azure Functions (Python 3.11)	REST API, business logic, webhooks	Staging + Production
Durable Functions	Azure Durable Functions	Long-running orchestrations (meeting bot processing)	Staging + Production
Primary Database	Azure Database for PostgreSQL	Relational data storage	Staging + Production
File Storage	Azure Blob Storage	Meeting recordings, transcripts, documents, uploads	Staging + Production

Secrets Management	Azure Key Vault	Credentials, API keys, certificates	Staging + Production
Monitoring	Azure Application Insights	Performance monitoring, logging, alerting	Staging + Production
DNS	Azure DNS / Custom domain provider	Domain management and routing	Production
SSL/TLS	Azure-managed certificates	HTTPS encryption	All

3.2 Environment Strategy

Environment	Purpose	Access	Deployment
Development	Local developer machines	Individual developers	Manual (`func start`, `npm run dev`)
Staging	Pre-production testing and QA	Engineering team	Automatic on merge to `main`
Production	Live user-facing environment	Restricted (Engineering Lead approval)	Manual promotion from staging

3.3 Architecture Diagram

```
                  ┌──────────────┐
                  │  Users/Admins  │
                  └──────┬───────┘
                         │ HTTPS
                  ┌──────▼───────┐
                  │ Azure Static  │
                  │  Web Apps     │
                  │ (Frontend)    │
                  └──────┬───────┘
                         │ HTTPS
                  ┌──────▼───────┐        ┌──────────────┐
                  │ Azure Functions │──────▶│ Azure Blob  │
                  │ (Backend API)  │       │  Storage    │
                  └──┬──┬──┬──────┘        └──────────────┘
                     │  │  │
            ┌────────┘  │  └────────┐
            ▼           ▼           ▼
┌──────────────┐ ┌──────────────┐ ┌──────────────┐
│ PostgreSQL   │ │ Key Vault    │ │ App Insights │
│ (Database)   │ │ (Secrets)    │ │ (Monitoring) │
└──────────────┘ └──────────────┘ └──────────────┘
```

---

4. Infrastructure Management

4.1 Provisioning

Principle	Implementation
Infrastructure as Code	Azure resource configurations documented and version-controlled
Consistent environments	Staging mirrors production configuration
Least privilege	Resources provisioned with minimum required permissions
Encryption by default	All new resources created with encryption enabled
Private by default	No public access unless explicitly required and documented

4.2 Configuration Management

Azure Functions (Backend):

Setting	Staging	Production
Runtime	Python 3.11	Python 3.11
HTTPS Only	Enabled	Enabled
Minimum TLS	1.2	1.2
Function Auth Level	Function keys	Function keys
CORS Origins	Staging frontend URL	Production frontend URL
App Settings Source	Azure Portal	Azure Key Vault references

Azure Static Web Apps (Frontend):

Setting	Staging	Production
Framework	Next.js 15	Next.js 15
HTTPS Only	Enforced (automatic)	Enforced (automatic)
Custom Domain	testapp.transparencyhubnetwork.ai	app.transparencyhubnetwork.ai
SSL Certificate	Azure-managed	Azure-managed

Azure Database for PostgreSQL:

Setting	Value
Version	Latest stable (16.x)
SSL Enforcement	Enabled (require SSL)
Firewall	Azure services only (no public access)
Encryption at Rest	Enabled (TDE)
Backup Retention	30 days

Geo-Redundant Backup	Enabled

Azure Blob Storage:

Setting	Value
Public Access	Disabled (private only)
Encryption	AES-256 SSE
Access Tier	Hot (frequently accessed files), Cool (archives)
Soft Delete	Enabled (7-day retention)
Firewall	Restricted to Azure Functions
Path Convention	`/{association_id}-{chapter_id}/{folder}/{year}/{month}/{day}/{file_name}`

4.3 Configuration Change Management

All infrastructure configuration changes must:

1. Be documented with justification before implementation
2. Be reviewed by at least one other engineer
3. Be tested in staging before applying to production
4. Be applied during low-traffic windows for production changes
5. Be verified after implementation
6. Be logged in the change management record

Emergency changes (during active incidents) may bypass the review process but must be documented and reviewed retrospectively within 48 hours.

4.4 Access Control

Resource	Access Method	Who	MFA Required
Azure Portal	Azure AD authentication	Engineering team	Yes
Azure Functions (management)	Azure Portal / CLI	Engineering team	Yes
Production Database	Azure Portal only (no direct connection)	Engineering Lead	Yes
Staging Database	Connection string via App Settings	Engineering team	Yes
Key Vault	Azure RBAC	Engineering Lead	Yes
Blob Storage (management)	Azure Portal / CLI	Engineering team	Yes

GitHub (deployments)	GitHub Actions (OIDC / deploy tokens)	CI/CD only	N/A

4.5 Resource Monitoring

Metric	Threshold	Alert Action
Function execution errors	> 5% error rate for 5 minutes	Engineering team notified
Function execution duration	> 30 seconds average	Investigate performance
Database CPU utilization	> 80% for 10 minutes	Scale up or optimize queries
Database storage usage	> 80% capacity	Plan capacity increase
Blob Storage capacity	> 80% capacity	Review retention, plan increase
HTTP 5xx responses	> 1% of requests	Investigate immediately
SSL certificate expiry	< 30 days	Renew (auto-managed by Azure)

---

5. Patching and Updates

5.1 Patching Strategy

Component	Responsibility	Patching Approach	Timeline
Azure platform (host OS, runtime)	Microsoft (PaaS)	Automatic	Managed by Azure
Azure Functions runtime	Shared (Microsoft + our config)	Monitor and update runtime version	Within 14 days of release
Python runtime	Engineering team	Update `requirements.txt` and runtime config	Within 14 days for security patches
Node.js runtime	Engineering team	Update `package.json` engines and CI config	Within 14 days for security patches
Application dependencies	Engineering team	Dependabot + manual review	Per severity SLA (see Section 6)
Custom application code	Engineering team	SAST/DAST findings + code reviews	Per vulnerability management SLA

5.2 Patching SLAs

Severity	Infrastructure Patches	Dependency Patches
Critical	Within 24 hours	Within 24 hours
High	Within 7 days	Within 7 days
Medium	Within 30 days	Within 30 days

Low	Next maintenance window	Within 90 days

5.3 Patching Process

1. **Notification** — Patch availability detected (Azure advisory, Dependabot, vendor notification)
2. **Assessment** — Evaluate applicability, severity, and potential impact
3. **Testing** — Apply patch in staging environment and verify functionality
4. **Approval** — Engineering Lead approves production deployment
5. **Deployment** — Apply patch to production during maintenance window (or immediately for Critical)
6. **Verification** — Confirm patch applied successfully, no regressions
7. **Documentation** — Record patch in change log

5.4 Maintenance Windows

Window	Schedule	Scope
Regular maintenance	Weekdays, 02:00–06:00 UTC	Non-critical patches, configuration changes
Emergency maintenance	Any time	Critical security patches, active incident response

Users are notified at least **24 hours** in advance for planned maintenance that may affect availability. Emergency maintenance proceeds without advance notice but with real-time status updates.

---

6. Dependency Management

6.1 Dependency Inventory

Backend (Python — Azure Functions):

Category	Examples	Package Manager	Lock File
Web Framework	Azure Functions SDK	pip	`requirements.txt` (pinned versions)
Database ORM	SQLAlchemy	pip	`requirements.txt`
Authentication	PyJWT, cryptography	pip	`requirements.txt`
AI/ML	google-genai	pip	`requirements.txt`
HTTP Client	requests, httpx	pip	`requirements.txt`
Testing	pytest, pytest-asyncio	pip	`requirements.txt`

Frontend (TypeScript — Next.js):

Category	Examples	Package Manager	Lock File
Framework	Next.js, React	npm	`package-lock.json`
State Management	Zustand	npm	`package-lock.json`
UI Components	Material UI, Tailwind CSS	npm	`package-lock.json`
Form Validation	Zod, React Hook Form	npm	`package-lock.json`

Rich Text Editor	react-quill-new	npm	`package-lock.json`
Date Handling	date-fns, date-fns-tz	npm	`package-lock.json`
HTTP Client	axios	npm	`package-lock.json`

6.2 Dependency Governance

Adding New Dependencies:

Before adding a new package, the developer must evaluate:

Criteria	Minimum Requirement
Maintenance	Active development (commits within last 6 months)
Popularity	npm: > 1,000 weekly downloads; PyPI: > 500 weekly downloads
Vulnerabilities	No unpatched Critical or High CVEs
License	Compatible with project (MIT, Apache 2.0, BSD preferred)
Size	Proportional to functionality needed (avoid bloat)
Alternatives	Evaluated at least one alternative or considered native implementation

New dependency additions must be documented in the PR description with justification.

Prohibited Dependencies:

- Packages with known unpatched Critical vulnerabilities
- Packages with restrictive licenses (GPL, AGPL) without legal review
- Packages that are abandoned (no commits in 12+ months, unresponsive maintainer)
- Packages that require unsafe installation flags (`--force` , `--legacy-peer-deps` without justification)

6.3 Version Pinning

Repository	Strategy	Example
Backend (Python)	Exact version pinning	`SQLAlchemy==2.0.35`
Frontend (Node.js)	Caret ranges with lock file	`"next": "^15.5.4"` + `package-lock.json`

- Lock files (`package-lock.json` , `requirements.txt`) are **always committed** to version control
- Lock files are the source of truth for installed versions
- Direct dependency ranges allow minor/patch updates; lock file controls exact resolution
- `npm ci` (not `npm install`) used in CI/CD for deterministic builds

6.4 Transitive Dependency Management

Transitive (indirect) dependencies are monitored through:

- **Dependabot** — Scans full dependency tree continuously
- **npm audit / pip audit** — Checks for known vulnerabilities in all resolved packages
- **npm overrides** — Forces safe versions of transitive dependencies when upstream fixes are unavailable

When overrides are used:

1. Override is documented in `package.json` with a comment explaining why
2. Upstream issue/PR is tracked for when the override can be removed
3. Overrides are reviewed monthly and removed when no longer needed

6.5 Automated Dependency Scanning

Tool	Scope	Frequency	Action
GitHub Dependabot	All repositories	Continuous	Creates PRs for vulnerable dependencies
Dependabot Version Updates	All repositories	Weekly	Creates PRs for outdated dependencies
npm audit	Frontend	CI pipeline (every PR)	Build fails on Critical/High vulnerabilities
pip audit	Backend	CI pipeline (every PR)	Build fails on Critical/High vulnerabilities
GitHub CodeQL	All repositories	Every PR + weekly	Detects vulnerable dependency usage patterns

6.6 Dependency Update Process

Automated (Dependabot PRs):

1. Dependabot creates PR with version bump
2. CI runs (lint, build, test, audit)
3. Developer reviews changelog and breaking changes
4. If CI passes and no breaking changes: merge
5. If breaking changes: assess impact, update code, then merge

Manual Updates:

1. Developer identifies needed update (security advisory, feature need)
2. Update version in `package.json` / `requirements.txt`
3. Run full test suite locally
4. Create PR with justification
5. CI validates, reviewer approves, merge

Major Version Upgrades:

1. Assess breaking changes from changelog/migration guide
2. Create feature branch for upgrade
3. Apply all required code changes
4. Run full test suite + manual testing
5. Deploy to staging for extended testing (minimum 48 hours)
6. Create PR with detailed migration notes
7. Deploy to production after staging verification

6.7 Dependency Vulnerability Response

Severity	Response	Timeline
Critical	Immediate update or override; emergency release if needed	24 hours

High	Update in next release cycle	7 days
Medium	Update in next sprint	30 days
Low	Bundle with next planned update	90 days
No fix available	Assess risk, implement compensating controls, evaluate alternatives	Document within 7 days

6.8 End-of-Life Dependencies

Status	Definition	Action	Timeline
Deprecated	Maintainer recommends migration to alternative	Plan migration	Within 90 days
Unmaintained	No updates in 12+ months, no response to issues	Evaluate alternatives, plan migration	Within 60 days
End of Life	Official end of support announced	Migrate to supported alternative	Before EOL date (or within 30 days)
Security EOL	No longer receiving security patches	Immediate migration planning	Within 30 days

---

7. CI/CD Pipeline

7.1 Pipeline Architecture

```
Developer Push / PR
       │
       ▼
┌──────────────┐
│    Lint      │  ESLint (frontend), ruff/flake8 (backend)
└──────┬───────┘
       ▼
┌──────────────┐
│    Build     │  Verify compilation, type checking
└──────┬───────┘
       ▼
┌──────────────┐
│    Test      │  Unit tests, integration tests
└──────┬───────┘
       ▼
┌──────────────┐
│  Security    │  npm audit, pip audit, secret scanning
│  Checks      │  CodeQL SAST analysis
└──────┬───────┘
       ▼
┌──────────────┐
```

```
│   Deploy      │  Staging (auto on main), Production (manual approval)
└───────────────┘
```

7.2 Pipeline Security

Control	Implementation
Secrets	Stored in GitHub Encrypted Secrets; never in workflow files
Permissions	Workflows use minimum required permissions
Dependencies	`npm ci` for deterministic installs; no `npm install` in CI
Pinned Actions	GitHub Actions pinned to specific versions or SHA
Branch Protection	PRs required for `main`; direct push blocked
Review Required	At least 1 approval before merge
Status Checks	All CI checks must pass before merge allowed

7.3 Deployment Process

Environment	Trigger	Approval	Rollback
Staging	Automatic on merge to `main`	None (CI passes)	Revert commit and redeploy
Production	Manual promotion	Engineering Lead	Revert to previous deployment

7.4 Rollback Procedures

Scenario	Action	Time to Rollback
Failed deployment	Automatic rollback to last known good	< 5 minutes
Post-deployment issue detected	Revert commit, redeploy from `main`	< 15 minutes
Data migration issue	Restore database from point-in-time backup	< 4 hours
Configuration error	Revert App Settings in Azure Portal	< 5 minutes

---

8. Disaster Recovery

8.1 Backup Strategy

System	Backup Method	Frequency	Retention	Storage Location
PostgreSQL Database	Azure automated backup	Continuous (point-in-time)	30 days	Azure (geo-redundant)
Blob Storage	Geo-redundant storage (GRS)	Real-time replication	Continuous	Azure paired region
Application Code	Git (GitHub)	Every commit	Indefinite	GitHub (US)

Infrastructure Config	Version-controlled documentation	Every change	Indefinite	GitHub
Secrets	Azure Key Vault (geo-replicated)	Real-time	Continuous	Azure (geo-redundant)

8.2 Recovery Objectives

System	RTO (Recovery Time)	RPO (Recovery Point)
Frontend	< 30 minutes	Zero (static, redeployed from Git)
Backend API	< 1 hour	Zero (stateless, redeployed from Git)
Database	< 4 hours	< 5 minutes (point-in-time recovery)
Blob Storage	< 2 hours	Near-zero (geo-redundant)
Secrets (Key Vault)	< 1 hour	Near-zero (geo-replicated)

8.3 Disaster Recovery Procedures

Scenario 1: Application Failure

1. Identify failing component from Application Insights
2. Attempt restart of Azure Function / Static Web App
3. If restart fails: redeploy from latest `main` branch
4. Verify functionality

Scenario 2: Database Corruption or Loss

1. Identify extent of corruption/loss
2. Initiate point-in-time restore to last known good state
3. Verify data integrity
4. Replay any lost transactions from application logs if possible
5. Notify users if data loss occurred

Scenario 3: Azure Region Outage

1. Assess Azure status page for estimated recovery
2. If extended outage (> 4 hours): initiate failover to paired region
3. Update DNS to point to failover region
4. Restore database from geo-redundant backup in paired region
5. Verify all services operational in new region

Scenario 4: Complete Compromise (Worst Case)

1. Revoke all credentials and access tokens
2. Provision new Azure resources in clean subscription
3. Deploy application from GitHub (known clean source)
4. Restore database from geo-redundant backup
5. Rotate all secrets in new Key Vault
6. Update DNS records
7. Conduct thorough security review before restoring user access

8.4 DR Testing

Test	Frequency	Scope
Database backup restoration	Quarterly	Restore to staging, verify data integrity
Application redeployment	Quarterly	Full redeploy from Git to staging
Secret rotation drill	Semi-annually	Rotate all secrets, verify no service disruption
Full DR simulation	Annually	Simulate region failure, execute full recovery

---

9. Capacity Management

9.1 Current Resource Allocation

Resource	Current Tier	Auto-Scale	Capacity Limit
Azure Functions	Consumption Plan	Yes (automatic)	200 instances
Static Web Apps	Standard	Yes (automatic)	CDN-backed (global)
PostgreSQL	General Purpose (2 vCores)	Manual scaling	32 vCores max
Blob Storage	General Purpose v2	Automatic	5 PiB
Key Vault	Standard	N/A	5,000 transactions/10s

9.2 Scaling Strategy

Trigger	Action	Timeline
Database CPU > 80% sustained	Evaluate query optimization; scale up if needed	Within 7 days
Database storage > 80%	Archive old data; increase storage	Within 7 days
API latency > 2s average	Profile and optimize; scale if needed	Within 14 days
Blob storage > 80%	Review retention policy; archive or expand	Within 14 days
User base doubles	Review all resource tiers and scaling configs	Proactive planning

9.3 Cost Management

- Azure spending monitored monthly
- Budget alerts configured at 80% and 100% of monthly budget
- Resource utilization reviewed quarterly to right-size allocations
- Unused or underutilized resources identified and decommissioned
- Cost allocation tagged by service component

---

10. Service Level Objectives

10.1 Availability Targets

Service	Target Availability	Measurement	Exclusions
Frontend (dashboard)	99.9%	Monthly uptime	Planned maintenance
Backend API	99.9%	Monthly uptime	Planned maintenance
Database	99.95%	Monthly uptime (Azure SLA)	Planned maintenance
Meeting bot scheduling	99.5%	Successful bot deployments / total requests	Third-party (Attendee.dev) outages
Payment processing	99.9%	Successful transactions / total attempts	Third-party (Stripe/Paystack) outages

10.2 Performance Targets

Metric	Target	Measurement
API response time (p50)	< 200ms	Application Insights
API response time (p95)	< 1 second	Application Insights
API response time (p99)	< 3 seconds	Application Insights
Frontend page load (p50)	< 2 seconds	Application Insights
Database query time (p95)	< 500ms	Application Insights

---

11. Third-Party Service Management

11.1 Service Inventory

Service	Purpose	Criticality	SLA	Fallback
Microsoft Azure	Core infrastructure	Critical	99.95% (composite)	Multi-region failover
Attendee.dev	Meeting bot recording	High	Best effort	Manual recording by participants
Google Gemini AI	Meeting minute generation	Medium	Google Cloud SLA	Manual minute creation
Stripe	Global payment processing	High	99.99%	Paystack (for supported regions)
Paystack	African payment processing	High	99.9%	Stripe (limited region support)
GitHub	Source code, CI/CD	High	99.9%	Local Git mirrors

Google OAuth	Google Meet provider auth	Medium	99.9%	Other providers (Zoom, Teams)
Zoom (via Attendee.dev)	Zoom meeting provider	Medium	99.9%	Other providers (Meet, Teams)
Microsoft Teams OAuth	Teams meeting provider	Medium	99.9%	Other providers (Meet, Zoom)

11.2 Vendor Monitoring

- Third-party status pages monitored for outage notifications
- Vendor SLA compliance reviewed quarterly
- Integration health checks run periodically
- Vendor incident impact assessed against our services
- Alternative vendors evaluated annually

11.3 Vendor Lock-In Mitigation

Strategy	Implementation
Standard protocols	OAuth 2.0, REST APIs, standard data formats
Abstraction layers	Service interfaces abstract vendor-specific implementations
Data portability	Data exportable in standard formats (JSON, CSV)
Multi-provider support	Meeting recording supports 3 providers; payment supports 2
Documentation	Integration architecture documented for migration planning

---

12. Compliance and Audit

12.1 Infrastructure Audits

Audit	Frequency	Scope
Azure configuration review	Quarterly	Security settings, firewall rules, access policies
Dependency audit	Monthly	Vulnerability scan, license review, EOL check
Access control review	Every 90 days	User permissions, service accounts, API keys
Backup verification	Quarterly	Test restoration, verify data integrity
CI/CD pipeline review	Semi-annually	Permissions, secrets, action versions
Cost and capacity review	Quarterly	Resource utilization, spending, scaling needs

12.2 Audit Trail

All infrastructure and dependency changes are logged:

- Azure Activity Log (all resource modifications)
- GitHub commit history (all code and config changes)

- GitHub Actions logs (all CI/CD executions)
- Key Vault access logs (all secret access)
- Change management records (all approved changes)

Audit logs retained for minimum **1 year**.

---

13. Related Documents

Document	Description
Security Policy	Comprehensive security controls and practices
SSDLC	Secure Software Development Lifecycle
SAST & DAST	Application security testing documentation
Vulnerability Management Policy	Vulnerability identification and remediation
Incident Management and Response Policy	Incident detection, response, and recovery
Data Retention and Protection Policy	Data handling, retention, and disposal

---

14. Document Control

Version	Date	Author	Changes
1.0	2026-03-05	Engineering Team	Initial version

This policy is reviewed and updated quarterly, or when significant infrastructure changes are made, new services are adopted, or security incidents reveal gaps.

For questions about this policy, contact: support@transparencyhubnetwork.ai